GenomicRanges - Lists

Kasper D. Hansen

Dependencies

This document has the following dependencies:

```
library(GenomicRanges)
```

Use the following commands to install these packages in R.

```
source("http://www.bioconductor.org/biocLite.R")
biocLite(c("GenomicRanges"))
```

Corrections

Improvements and corrections to this document can be submitted on its GitHub in its repository.

Overview

In this session we will discuss `GRangesList` which is a list of `GRanges` (whoa; blinded by the insight here …).

Other Resources

Why

The *IRanges* and *GenomicRanges* packages introduced a number of classes I'll call `XXList`; an example is `GRangesList`.

These looks like standard `lists` from base R, but they require that every element of the list is of the same class. This is convenient from a data structure perspective; we know exactly what is in the list.

But things are also happening behind the scenes. These types of lists often have additional compression build into them. Because of this, it is best to use specific methods/functions on them, as opposed to the standard toolbox of `sapply`/`lapply` that we use for normal lists. This will be clearer below.

An important usecase specifically for `GRangesList` is the representation of a set of **transcripts**. Each transcript is an element in the list and the **exons** of the transcript is represented as a `GRanges`.

GrangesList

Let us make a `GRangesList`:

```
gr1 <- GRanges(seqnames = "chr1", ranges = IRanges(start = 1:4, width =
gr2 <- GRanges(seqnames = "chr2", ranges = IRanges(start = 1:4, width =
gL <- GRangesList(gr1 = gr1, gr2 = gr2)
gL
```

```
## GRangesList object of length 2:
## $gr1
## GRanges object with 4 ranges and 0 metadata columns:
##       seqnames    ranges strand
##          <Rle> <IRanges>  <Rle>
##   [1]     chr1    [1, 3]      *
##   [2]     chr1    [2, 4]      *
##   [3]     chr1    [3, 5]      *
##   [4]     chr1    [4, 6]      *
##
## $gr2
## GRanges object with 4 ranges and 0 metadata columns:
##       seqnames ranges strand
##   [1]     chr2 [1, 3]      *
##   [2]     chr2 [2, 4]      *
##   [3]     chr2 [3, 5]      *
##   [4]     chr2 [4, 6]      *
##
## -------
## seqinfo: 2 sequences from an unspecified genome; no seqlengths
```

A number of standard `GRanges` functions work, but returns (for example) `IntegerLists`

```
start(gL)
```

```
## IntegerList of length 2
## [["gr1"]] 1 2 3 4
## [["gr2"]] 1 2 3 4
```

```
seqnames(gL)
```

```
## RleList of length 2
## $gr1
## factor-Rle of length 4 with 1 run
##   Lengths:    4
##   Values : chr1
## Levels(2): chr1 chr2
##
## $gr2
## factor-Rle of length 4 with 1 run
##   Lengths:    4
##   Values : chr2
## Levels(2): chr1 chr2
```

I very often want to get the length of each of the elements. Surprisingly it is very slow to get this using `sapply(gL, length)` (or at least it used to be very slow). There is a dedicated function for this:

```
elementLengths(gL)
```

```
## gr1 gr2
##   4   4
```

We have a new `XXapply` function with the fancy name `endoapply`. This is used when you want to apply a function which maps a `GRanges` into a `GRanges`, say a `shift` or `resize`.

```
shift(gL, 10)
```

```
## GRangesList object of length 2:
## $gr1
## GRanges object with 4 ranges and 0 metadata columns:
##       seqnames    ranges strand
##          <Rle> <IRanges>  <Rle>
##   [1]     chr1  [11, 13]      *
##   [2]     chr1  [12, 14]      *
##   [3]     chr1  [13, 15]      *
##   [4]     chr1  [14, 16]      *
##
## $gr2
## GRanges object with 4 ranges and 0 metadata columns:
##       seqnames   ranges strand
##   [1]     chr2 [11, 13]      *
##   [2]     chr2 [12, 14]      *
##   [3]     chr2 [13, 15]      *
##   [4]     chr2 [14, 16]      *
##
## -------
## seqinfo: 2 sequences from an unspecified genome; no seqlengths
```

`findOverlaps` works slightly different. For `GRangesLists`, we think of each element is a union of ranges. So we get an overlap if any range overlaps. Lets us see

```
findOverlaps(gL, gr2)
```

```
## Hits object with 4 hits and 0 metadata columns:
##       queryHits subjectHits
##       <integer>   <integer>
##   [1]         2           1
##   [2]         2           2
##   [3]         2           3
##   [4]         2           4
##   -------
##   queryLength: 2
##   subjectLength: 4
```

Note how the `queryLength` is 2 and not 20. What we know from the first row of this output is that some range in `gL[[2]]` overlaps the range `gr[1]`.

This is actually a feature if we think of the `GRangesList` as a set of transcript, where each `GRanges` gives you the exon of the transcript. With this interpretation, `findOverlaps` tells you whether or not the **transcript** overlaps some region of interest, and this is true if any of the

exons of the transcript overlaps the region.

Other Lists

There are many other types of `XXList`, including

- `RleList`
- `IRangesList`
- `IntegerList`
- `CharacterList`
- `LogicalList`

and many others.

SessionInfo

```
## R version 3.2.1 (2015-06-18)
## Platform: x86_64-apple-darwin13.4.0 (64-bit)
## Running under: OS X 10.10.5 (Yosemite)
##
## locale:
## [1] en_US.UTF-8/en_US.UTF-8/en_US.UTF-8/C/en_US.UTF-8/en_US.UTF-8
##
## attached base packages:
## [1] stats4    parallel  methods   stats     graphics  grDevices utils
## [8] datasets  base
##
## other attached packages:
## [1] GenomicRanges_1.20.5 GenomeInfoDb_1.4.2   IRanges_2.2.7
## [4] S4Vectors_0.6.3      BiocGenerics_0.14.0  BiocStyle_1.6.0
## [7] rmarkdown_0.7
##
## loaded via a namespace (and not attached):
##  [1] digest_0.6.8    formatR_1.2     magrittr_1.5    evaluate_0.7.2
##  [5] stringi_0.5-5   XVector_0.8.0   tools_3.2.1     stringr_1.0.0
##  [9] yaml_2.1.13     htmltools_0.2.6 knitr_1.11
```